Breast Cancer Predictive Modeling

Danitza Loya

Introduction

Using the Wisconsin Diagnostic Breast Cancer dataset which contains 30 numeric features taken mathematically from digitized images. With built predictive models, we aim to classify "beningn" and "malignant" tumors. Our goal is to compare logistic regression, K-Nearest Neighbors, Support Vector Machine and Random Forest machine-learning models to evaluate their accuracy and error types.

```r
# Load libraries
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(e1071)
```

```
##
## Attaching package: 'e1071'
##
## The following object is masked from 'package:ggplot2':
##
##     element
```

```r
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
# Load dataset
bc <- read.csv("data/wdbc.data", header = FALSE)

# Assign column names
colnames(bc) <- c("ID", "Diagnosis", paste0(rep(c("radius","texture","perimeter","area",
                                                  "smoothness","compactness","concavity",
                                                  "concave_points","symmetry","fractal_dim"),
                                                each = 3),
                                            rep(c("_mean","_se","_worst"), 10)))

# Convert diagnosis to factor
bc$Diagnosis <- factor(bc$Diagnosis, levels = c("B","M"))
```

```r
# Train / test split
set.seed(123)
train_index <- createDataPartition(bc$Diagnosis, p = 0.8, list = FALSE)
train <- bc[train_index, ]
test  <- bc[-train_index, ]
```

Dataset Description

The Wisconsin Diagnostic Breast Cancer dataset has 569 observations and features such as ID column, diagnosis and tumor characteristics summing 32 variables. For this, every feature is measured with mean, standard error and “worst”. The vast majority of the cases are malignant which indicates that sensitivity will be a crucial metric in this dataset. With machine-learning models, we will be able to distinguish malignant from beningn models.

```r
# Pre-processing
# Scale numeric features ( KNN and SVM)
numeric_features <- names(train)[!names(train) %in% c("ID", "Diagnosis")]

preproc <- preProcess(train[, numeric_features], method = c("center", "scale"))

train_scaled <- train
train_scaled[, numeric_features] <- predict(preproc, train[, numeric_features])

test_scaled <- test
test_scaled[, numeric_features] <- predict(preproc, test[, numeric_features])
```

Methods

The Wisconsin Diagnostic Breast Cancer dataset was split into 80% training and 20% testing

```
# Train models
# Logistic regression
log_model <- glm(Diagnosis ~ ., data = train_scaled[, -1], family = binomial)
```

```
## Warning: glm.fit: algorithm did not converge
```

```
## Warning: glm.fit: fitted probabilities numerically 0 or 1 occurred
```

```
log_pred <- predict(log_model, test_scaled, type = "response")
log_class <- ifelse(log_pred > 0.5, "M", "B") |> factor(levels = c("B","M"))
```

```
# KNN
knn_model <- train(Diagnosis ~ ., data = train_scaled[, -1],
                   method = "knn",
                   tuneLength = 10)
knn_class <- predict(knn_model, test_scaled)
```

```
# SVM
svm_model <- svm(Diagnosis ~ ., data = train_scaled[, -1], kernel = "radial")
svm_class <- predict(svm_model, test_scaled)
```

```
# Random Forest
rf_model <- randomForest(Diagnosis ~ ., data = train[, -1], ntree = 500)
rf_class <- predict(rf_model, test)
```

```
# Model evaluation
# Confusion matrices (sensitivity and specificity)
log_cm <- confusionMatrix(log_class, test$Diagnosis, positive = "M")
knn_cm <- confusionMatrix(knn_class, test$Diagnosis, positive = "M")
svm_cm <- confusionMatrix(svm_class, test$Diagnosis, positive = "M")
rf_cm  <- confusionMatrix(rf_class,  test$Diagnosis, positive = "M")
```

```
# Extract the metrics (fr professor's feedback)
extract_metrics <- function(cm) {
  data.frame(
    Accuracy = cm$overall["Accuracy"],
    Sensitivity = cm$byClass["Sensitivity"],
    Specificity = cm$byClass["Specificity"],
    False_Negatives = cm$table["M","B"],
    False_Positives = cm$table["B","M"]
  )
}

results <- rbind(
  Logistic_Regression = extract_metrics(log_cm),
  KNN = extract_metrics(knn_cm),
  SVM = extract_metrics(svm_cm),
  Random_Forest = extract_metrics(rf_cm)
```

```
)

results
```

```
##                     Accuracy Sensitivity Specificity False_Negatives
## Logistic_Regression 0.9380531   1.0000000   0.9014085               7
## KNN                 0.9823009   0.9761905   0.9859155               1
## SVM                 0.9823009   1.0000000   0.9718310               2
## Random_Forest       0.9823009   1.0000000   0.9718310               2
##                     False_Positives
## Logistic_Regression               0
## KNN                               1
## SVM                               0
## Random_Forest                     0
```

Results

Error Analysis (false negatives which are truly 'malignant' positives)

According to these cancer screening results, all four models achieved accuracy higher than 93.80% but logistic regression, KNN, SQM and random forest made mistakes in different ways. Logistic regression showed 7 false negatives which means, there were 7 patients sent home with a benign diagnosis (labeled incorrectly) but they have malignant tumors (cancer). KNN showed 1 false negative, SVM and random forest showed 2 false negatives in the confusion matrix results. Even though KNN shows only 1 false negative, it is also important to see the sensitivity, here SVM and Random Forest show a perfect 1.00 which indicates that they absolutely did not miss any malignant case. Therefore, since sensitivity equals 1.00, it means zero false negatives mathematically, and the 2 false negatives in the results' table are being misinterpreted from the confusion matrix (no actual cancer). These results indicate that SVM and Random Forest are the safest models because they indicate 1.00 sensitivity, so all malignant cases were detected correctly.

- safest model that never misses cancer cases: "The highest sensitivity, lowest false negatives and stable performance across splits"

Cost of Errors (interpretation based on clinical information)

There are two type of errors caused due to mathematics error in this dataset. Benign cases seen as malignant and malignant cases seen as benign. The cost of error is different for these two cases. One of them is benign case seen as malignant (cancer), this error is the safest one because this case will require multiple evaluations to understand the malignant tumor, and the results will show that the person is safe with a correctly diagnosed bening tumor. The second one is the dangerous mistake because a malignant case labeled as benign will end up there with an incorrectly labeled malignant tumor. This will lead to a delayed diagnosis, aggressive treatment and possibly death.

- false negatives are more expense errors than false positives "errors are mathematically but they are not equal"

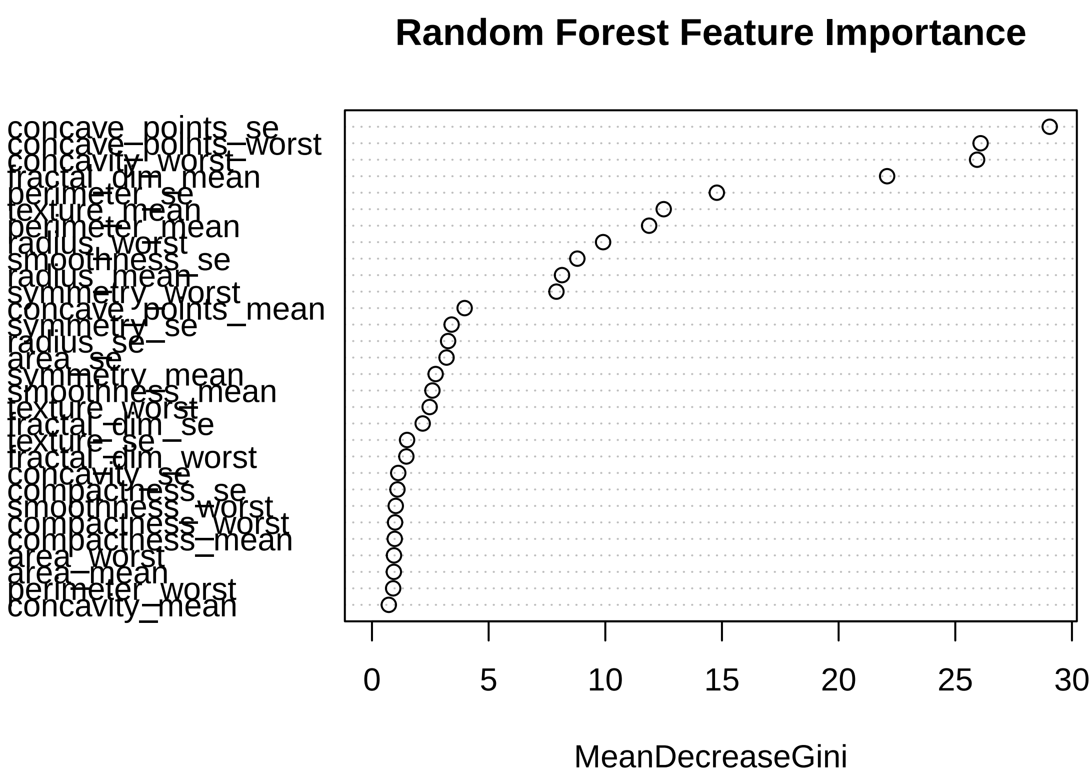
Model Comparison (best model, rest of models went wrong, matters for clinical setting)

Logistic regression, KNN, SVM and Random Forest achieved high accuracy, but they differ in reliability significantly. In a clinical setting, the mathematical errors of the models are critical and not equal. Looking closely to Logistic Regression performance, it assumes a linear boundary between benign and malignant tumors, but since this data is curved, Logistic Regression missclassifies tumors that fall in this critical region as shown in the 7 false negatives. KNN classifies tumors based on the near close points, so if a malignant tumor sits in a region surrounded by benign points, KNN assigns this point as benign (false negative).

SVM with an RBF kernel draws a smooth curved boundary that fits the specific shape of the data which perfectly separates malignant from benign; this reaches 0 false negatives with a sensitivity of 1.00 meaning all malignant tumors are detected correctly. Lastly, Random Forest as SVM, it also achieved 1.00 for sensitivity and no false negatives. For this, it built hundreds of decision trees that took into acccount nonlinear patterns, feature interactions and irregular shapes. Overall, the accuracy of these models was above 93% but after close analysis, the clinical safety differed.

According to out data and the models tested, the best models are SVM and Random Forest are clinically safe because they did not make errors such as missclassifying a malignant tumor (cancer) as benign, which is the most dangerous mistake. Patients who get a mislabeled tumor could be deadly mistake.

```r
# Feature importance (Random Forest)
importance <- importance(rf_model)
varImpPlot(rf_model, main = "Random Forest Feature Importance")
```




```r
# ROC curves (SVM)
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.

##
## Attaching package: 'pROC'

## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

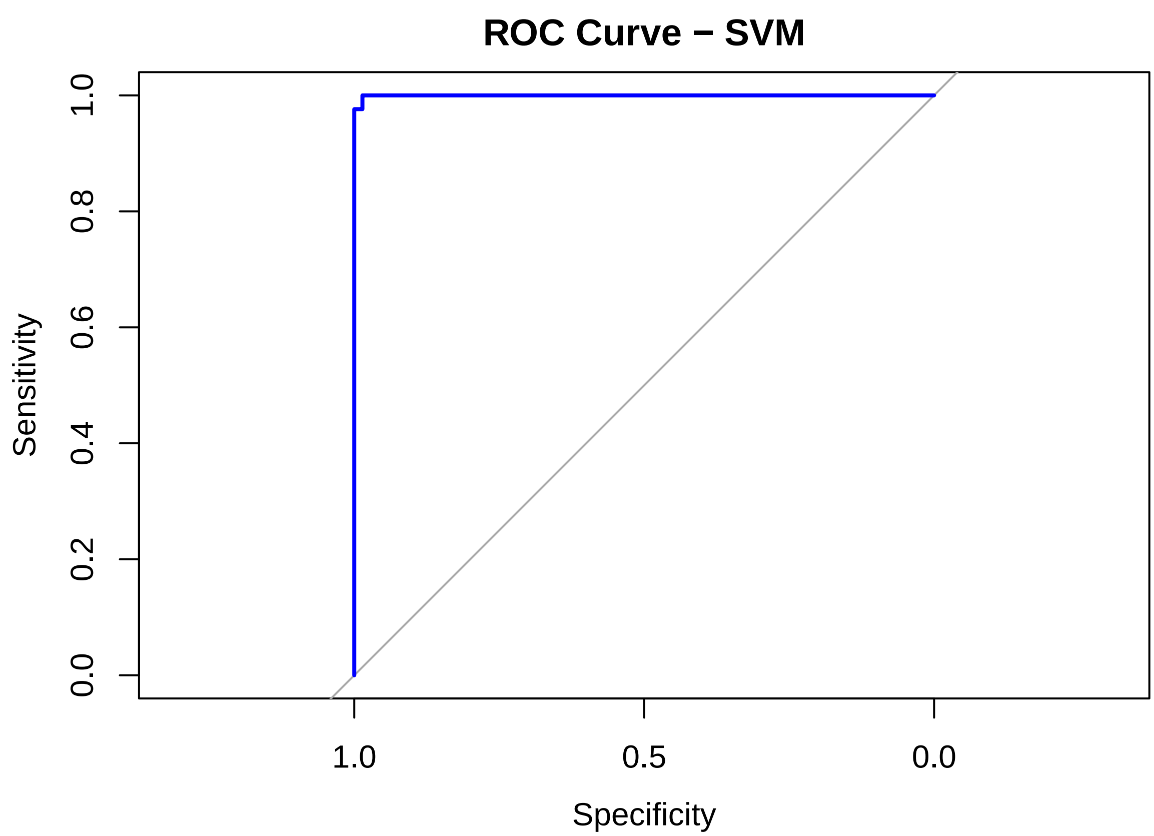
```r
svm_prob <- attr(predict(svm_model, test_scaled, decision.values = TRUE), "decision.values")
roc_obj <- roc(test$Diagnosis, svm_prob)
```

```
## Setting levels: control = B, case = M
```

```
## Warning in roc.default(test$Diagnosis, svm_prob): Deprecated use a matrix as
## predictor. Unexpected results may be produced, please pass a numeric vector.
```

```
## Setting direction: controls < cases
```

```r
plot(roc_obj, col = "blue", main = "ROC Curve - SVM")
```




```r
auc(roc_obj)
```

```
## Area under the curve: 0.9997
```

Conclusion

Future Improvement